

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

TP1 - Problema do Caixeiro Viajante

BCC202 - Estrutura de Dados I

Gustavo Ferreira e Marcelo Vieira
Professor: Pedro Silva

Ouro Preto
6 de novembro de 2023

Sumário

1	Introdução	1
1.1	Especificações do problema	1
1.2	Considerações iniciais	1
1.3	Ferramentas utilizadas	1
1.4	Especificações da máquina	1
1.5	Instruções de compilação e execução	2
2	Desenvolvimento	3
2.1	O Tipo Abstrato Definido	3
2.2	Alocando memória e lendo os dados	3
2.3	Imprimindo os resultados	3
2.4	Encontrando o melhor caminho	4
2.5	Implementação da função principal	5
3	Experimetos	6
4	Resultados	7
5	Considerações Finais	8

Lista de Figuras

1	Resultado da comparação entre saídas + o tempo de execução do programa.	7
---	---	---

Lista de Códigos Fonte

1	Implementação da struct GrafoPon.	3
2	Implementação da função imprimeCaminho.	3
3	Protótipo da função encontraCaminho.	4
4	Implementação da função encontraCaminho.	4
5	Implementação da função main.	5

1 Introdução

O problema do caixeiro viajante é antigo na Ciência da Computação, e consiste em determinar a melhor rota para percorrer uma série de cidades, retornando à cidade de origem. Para realizar esse trabalho, foram desenvolvido um algoritmo em C e um relatório referente ao que foi implementado. A codificação foi feita utilizando somente a biblioteca padrão da GNU, sem a utilização de bibliotecas adicionais.

1.1 Especificações do problema

Para resolver o problema do caixeiro viajante, foi definido previamente que uma função recursiva deve ser utilizada a fim de encontrar a menor distância possível entre as cidades. Além disso, um Tipo Abstrato de Dados (TAD) chamado GrafoPonderado também precisou ser implementado como forma de representar o caminho a ser analisado, ele possui dois atributos pré definidos: número de cidades a serem visitadas e uma matriz de adjacências. O TAD também deve implementar pelo menos 5 operações:

1. `alocarGrafo`: aloca um (ou mais) TAD GrafoPonderado.
2. `desalocarGrafo`: desaloca um TAD GrafoPonderado.
3. `leGrafo`: inicializa o TAD GrafoPonderado a partir de dados do terminal.
4. `encontraCaminho`: função recursiva que retorna o menor caminho no grafo fornecido.
5. `imprimeCaminho`: imprime na tela o menor caminho e a distância total percorrida.

1.2 Considerações iniciais

Algumas ferramentas foram utilizadas durante a criação deste projeto:

- Ambiente de desenvolvimento do código fonte: Visual Studio Code. ¹
- Linguagem utilizada: C.
- Ambiente de desenvolvimento da documentação: Overleaf L^AT_EX. ²

1.3 Ferramentas utilizadas

Algumas ferramentas foram utilizadas para testar a implementação, como:

- *Valgrind*: ferramentas de análise dinâmica do código.

1.4 Especificações da máquina

A máquina onde o desenvolvimento e os testes foram realizados possui a seguinte configuração:

- Processador: Ryzen 5 5600G.
- Memória RAM: 16Gb.
- Sistema Operacional: Linux.

¹Visual Studio Code está disponível em <https://code.visualstudio.com/>

²Disponível em <https://www.overleaf.com/>

1.5 Instruções de compilação e execução

Para a compilação do projeto, basta digitar:

Compilando o projeto

```
gcc -g -o exe arquivo1.c arquivo2.c -Wall
```

Usou-se para a compilação as seguintes opções:

- *-o exe*: para definir o nome do arquivo executável gerado.
- *-g*: para compilar com informação de depuração e ser usado pelo Valgrind.
- *-Wall*: para mostrar todos os possível *warnings* do código.

Para a execução do programa basta digitar:

```
./exe caminho_até_o_arquivo_de_entrada
```

2 Desenvolvimento

Nessa seção serão descritos os passos realizados para a implementação do programa, incluindo as dificuldades encontradas e as soluções definidas.

2.1 O Tipo Abstrato Definido

A fim de atender aos requisitos impostos, um tipo de dados chamado GrafoPon foi definido. O Código 5 apresenta o código implementado para a struct GrafoPon em C, contendo os atributos e suas devidas explicações em forma de comentários.

```
1 typedef struct grafoPon {
2     int numCidades; //numero de cidades totais
3     int **matrizAdjacencias; //matriz que armazena as distancias entre cada
      cidade
4     int *cidadeVisitada; //vetor de 0 e 1 que nos diz se uma cidade foi
      visitada ou nao
5     int *caminho; //vetor que armazena o melhor caminho pro dado grafo, uma
      vez que calculado
6     int *caminhoTemp; //vetor que armazena os caminhos em tempo de teste
7     int distancia; //inteiro que armazena a melhor distancia encontrada
8 } GrafoPon;
```

Código 1: Implementação da struct GrafoPon.

2.2 Alocando memória e lendo os dados

Com o tipo abstrato definido, o próximo passo consiste em alocar dinamicamente a memória para os atributos necessários e realizar a leitura dos dados. Para isso, foram criadas duas funções: "leGrafo" e "alocarGrafo". A função para ler os grafos primeiramente recebe o número de cidades que o caixeiro viajante precisará visitar, e logo em seguida chama a função responsável por alocar espaço para que a matriz de adjacências seja devidamente alocada. Essa matriz será de tamanho $n \times n$, onde n representa o número de cidades a serem visitadas. A função também aloca espaço para os três vetores da struct GrafoPon: cidadeVisitada, caminhoTemp e caminho. A melhor distância encontrada também é definida utilizando o parâmetro INT_MAX, da biblioteca limits.h, que define a melhor distância como sendo o maior número inteiro permitido pelo sistema. Após o espaço ser alocado o algoritmo volta para a função de leitura, que lê e armazena as distâncias entre as cidades no seguinte padrão: sendo i uma cidade A e j uma cidade B, um elemento $[i][j]$ da matriz representa a distância entre as cidades A e B.

Logo após a implementação das funções para leitura de dados e alocação de memória, também foi implementada a função "desalocarGrafo", que é chamada ao final da execução do algoritmo e responsável por desalocar a memória de todos os atributos que foram alocados dinamicamente na função "alocarGrafo".

Com isso, três das operações pré definidas já se encontravam implementadas.

2.3 Imprimindo os resultados

Após a implementação de três das cinco funções necessárias para o funcionamento do programa, foi decidido que o próximo passo seria implementar a função que imprime os resultados. Por ser uma função simples cujos parâmetros já são conhecidos, foi possível realizar sua implementação antes mesmo de definir a função que encontra o melhor caminho. O código 5 apresenta a implementação da função "imprimeCaminho".

```
1 void imprimeCaminho(GrafoPon *grafo) {
2     //imprime numa primeira linha o menor caminho a ser percorrido
3     for (int i = 0; i < grafo->numCidades; i++) {
4         printf("%d ", grafo->caminho[i]);
5     }
6     //como a ultima cidade sempre sera o 0, imprimimos ele ao final
7     printf("0 ");
```

```

8
9 //imprime numa segunda linha a distancia percorrida por esse menor caminho
10 printf("\n%d\n", grafo->distancia);
11 }

```

Código 2: Implementação da função imprimeCaminho.

2.4 Encontrando o melhor caminho

Com quatro das cinco operações implementadas, partimos para a função "encontraCaminho", responsável por encontrar o menor caminho possível entre as cidades na qual o caixeiro viajante deve passar. Inicialmente, foi utilizado um algoritmo de heap dentro de um laço for que realizava todas as permutações de n objetos, porém, o algoritmo não se mostrou a melhor saída quando combinado à estrutura definida e nem todas as permutações estavam sendo realizadas. Assim, a fim de conseguir gerar todas as permutações dentro do laço for de maneira simplificada, foram utilizados parâmetros inteiros para substituir as permutações por heap.

O código 5 apresenta o protótipo da função encontraCaminho.

```

1 void encontraCaminho(GrafoPon *grafo, int posicao, int contador, int
  distanciaTemp);

```

Código 3: Protótipo da função encontraCaminho.

O primeiro parâmetro que a função recebe é um ponteiro que apontada para o grafo do algoritmo, ele será importante para termos acesso ao número de cidades e à matriz de adjacências. Além disso, com acesso direto ao grafo conseguimos atualizar a qualquer momento a melhor distância, o vetor contendo o melhor caminho, o vetor contendo os caminhos temporários definidos e o vetor que informa qual cidade foi visitada. O parâmetro posição informa a cidade atual que o caixeiro viajante está visitando, e como ele parte da primeira cidade, sempre iremos iniciar a função passando o número 0 como parâmetro. O contador é o parâmetro que conta o número de cidades visitadas até o momento, ele deve ser inicializado como 1, pois o caixeiro já visitou a primeira cidade. O último parâmetro, chamado de distanciaTemp, irá armazenar a distância calculada atualmente, que começa em 0 e vai sendo incrementado conforme o caixeiro visita outras cidades.

O código 5 apresenta a implementação completa da função encontraCaminho e como os parâmetros são utilizados.

```

1 void encontraCaminho(GrafoPon *grafo, int posicao, int contador, int
  distanciaTemp) {
2   if ((contador == grafo->numCidades) && (grafo->matrizAdjacencias[posicao]
    [0])) {
3     distanciaTemp += grafo->matrizAdjacencias[posicao][0];
4     if (distanciaTemp < grafo->distancia) {
5       grafo->distancia = distanciaTemp;
6       for (int k = 0; k < grafo->numCidades; k++)
7         grafo->caminho[k] = grafo->caminhoTemp[k];
8     }
9     return;
10  }
11  for (int i = 0; i < grafo->numCidades; i++) {
12    if (grafo->cidadeVisitada[i] == 0 && grafo->matrizAdjacencias[posicao]
      [i]) {
13      grafo->cidadeVisitada[i] = 1;
14      grafo->caminhoTemp[contador] = i;
15      encontraCaminho(grafo, i, contador + 1, distanciaTemp + grafo->
        matrizAdjacencias[posicao][i]);
16      grafo->cidadeVisitada[i] = 0;
17    }
18  }
19 }

```

Código 4: Implementação da função encontraCaminho.

Primeiramente, a função irá conferir se todas as cidades já foram visitadas pelo caixeiro viajante no caminho atual e se a distância entre a cidade atual e a inicial é diferente de 0. O algoritmo inicialmente desenvolvido apenas conferia se todas as cidades haviam sido visitadas, não conferindo se a distância entre a cidade atual e inicial era diferente de 0, porém, em determinados casos de teste essa condição se tornou necessária, pois algumas vezes determinadas cidades não possuem ligação com a cidade inicial.

Caso as duas condições sejam satisfeitas, a distância total do caminho atual é calculada, e caso ela seja menor do que a menor distância encontrada até o momento, atualizamos a menor distância para receber a distância do caminho atual. Além de atualizar a menor distância, também é necessário atualizar o vetor que armazena a ordem das cidades do caminho para que receba a ordem das cidades do caminho atual.

O laço for irá iterar sobre todas as cidades, caso a cidade da iteração não tenha sido visitada e haja uma distância diferente de 0 (o que significa ser possível viajar entre elas) entre ela e a cidade atual, a cidade da iteração é visitada e seu número é inserido no vetor que armazena o caminho temporário. A função é então chamada novamente para que a próxima cidade seja visitada, com parâmetros atualizados:

1. Posição recebe o número da iteração, que representa a próxima cidade a ser visitada;
2. O contador é incrementado, pois o caixeiro acaba de visitar uma cidade;
3. A distância do caminho atual é incrementada, sendo somada com a distância entre a cidade atual e a cidade da iteração;

Após a chamada recursiva, a cidade deve ser marcada como não visitada para que as próximas permutações possam ser realizadas normalmente.

2.5 Implementação da função principal

Com todo o TAD definido, resta criar um grafo na função principal e realizar as operações na ordem correta. O código 5 apresenta a implementação da função main e as chamadas de funções.

```
1 int main() {  
2  
3     GrafoPon grafo;  
4  
5     leGrafo(&grafo);  
6  
7     encontraCaminho(&grafo, 0, 1, 0);  
8  
9     imprimeCaminho(&grafo);  
10  
11    desalocarGrafo(&grafo);  
12  
13    return 0;  
14 }
```

Código 5: Implementação da função main.

3 Experimentos

Após implementação do código, uma série de experimentos foram feitos testando diferentes entradas. Os números de cidades foram testados conforme arquivos de testes enviados, utilizando a IDE Visual Studio Code e o terminal do Linux. O arquivo de testes é composto de 5 entradas, e o número de cidades definido como 4, 6, 4, 5 e 5. Dois dos cinco arquivos possuíam cidades cuja distância uma para a outra era de 0, ou seja, não há como o caixeiro viajante ir de uma para a outra. O tempo de execução do arquivo de testes também foi testado.

4 Resultados

O algoritmo se mostrou eficiente para a resolução de todos os casos de testes, conforme mostrado na Figura 1.

```
gustavo@gustavo-VirtualBox:~/Downloads/TP1TESTES$ time python3 corretor.py
Analisando atividade:
  1.in OK
  2.in OK
  3.in OK
  4.in OK
  5.in OK
Nota na atividade: 10.00

real    0m0,257s
user    0m0,145s
sys     0m0,091s
gustavo@gustavo-VirtualBox:~/Downloads/TP1TESTES$
```

Figura 1: Resultado da comparação entre saídas + o tempo de execução do programa.

5 Considerações Finais

Após a realização do trabalho, foi possível compreender melhor sobre a utilização de alocação dinâmica, structs e funções recursivas. Também foi possível aprender sobre processos para debugar um programa, tendo em vista que durante o desenvolvimento do algoritmo vários bugs foram encontrados.